1:N Face Recognition Platform

Comprehensive architecture and implementation documentation for the enrollment and identification services.

Generated: 2026-07-09 15:17:37

1:N Face Recognition Platform - Technical Documentation

Version: 1.0.0

Generated on: 2026-07-09

1. Overview

This repository implements a two-service 1:N face recognition platform:

- `enrollment_service`: registers candidates and generates 512-dimensional normalized facial embeddings.
- `face_identification`: identifies unknown face images against enrolled candidates using FAISS vector search.

Both services are built with FastAPI, async SQLAlchemy, PostgreSQL persistence, and InsightFace-based computer vision pipelines. Streamlit applications are included for operator workflows.

2. Repository Structure

- `enrollment_service/`
  - `app/`: API, business services, repositories, schemas, recognition pipeline.
  - `streamlit_ui/`: enrollment and data-management operator dashboard.
  - `scripts/`: setup, run, lint, and test scripts.
  - `docker/` and `docker-compose.yml`: reference container setup.
- `face_identification/`
  - `app/`: identification API, recognition modules, FAISS management, metrics.
  - `streamlit_app/`: live identification and index/import dashboard.
  - `scripts/`: local bootstrap scripts.

3. High-Level Architecture

3.1 Logical Layers

Both services follow a similar layered pattern:

- Presentation: FastAPI routers, middleware (logging, rate limiting, CORS), validation and error mapping.
- Application: orchestration services (`EnrollmentService`, import services, candidate services).
- Domain: Pydantic request/response schemas and custom exceptions.
- Infrastructure: SQLAlchemy repositories, InsightFace processing components, FAISS index (identification service).

3.2 Runtime Flow (End-to-End)

1. Candidate enrollment request enters `enrollment_service`.
2. Image bytes are validated and decoded.
3. Face pipeline performs detection, quality checks, alignment, and ArcFace embedding extraction.
4. Candidate profile and embedding vectors are stored in PostgreSQL.
5. `face_identification` imports/syncs embeddings and rebuilds FAISS index.
6. Identification requests generate query embeddings and run vector search.
7. Top match is validated against similarity threshold and resolved to candidate metadata.

4. Enrollment Service (`enrollment_service`)

4.1 Purpose

Enroll candidates and produce high-quality, normalized embeddings for downstream identification.

4.2 Application Startup

`app/main.py` creates the FastAPI app and configures:

- SlowAPI rate limiting middleware and exception handling.
- Request logging middleware.
- CORS (`allow_origins=["*"]`).
- Unified JSON error envelope for application, validation, HTTP, and unexpected exceptions.

`app/core/events.py` lifecycle:

- Ensures export directory exists.
- Loads `FaceRecognitionPipeline` on startup.
- Stores pipeline instance in `app.state.pipeline`.

4.3 Recognition Pipeline

`app/recognition/pipeline.py` pipeline steps:

1. Detect faces using InsightFace `FaceAnalysis` (SCRFD detector from `buffalo_l`).
2. Run quality checks with `CompositeQualityChecker`.
3. Align face using `insightface.utils.face_align.norm_crop` to `112x112`.
4. Generate ArcFace embedding and L2-normalize.

Output includes:

- 512-d embedding vector.
- model name (`buffalo_l/w600k_r50`).
- quality outcome and timing breakdown (`detection`, `quality`, `alignment`, `embedding`).

4.4 Quality Enforcement

Quality checks reject images that fail conditions such as:

- no face or multiple faces,
- low detector confidence,
- face bounding box too small,
- blur,
- too dark or too bright,
- very low image resolution.

4.5 Persistence and Data Model

Database session management (`app/database/engine.py`):

- async engine (`postgresql+asyncpg`),
- request-scoped session dependency with commit/rollback safety.

Core tables (from models and README):

- `candidates`:
  - UUID primary key, business `candidate_id`, `name`, `email`, optional `dob`, audit timestamps.
- `face_embeddings`:
  - UUID primary key, FK to candidate UUID, float array embedding, dimension, model metadata, normalization flag.

4.6 Enrollment API Surface

Base prefix: `/api/v1`

- `GET /health`
- Candidate CRUD under `/candidates`
- Face enrollment:
  - `POST /face/upload` (multipart form + image)
  - `POST /face/capture` (base64 payload)
- Embedding operations:
  - generate/regenerate/list endpoints in `/embedding/*`
- Export:
  - `GET /export/json`
  - `GET /export/csv`
- Import:
  - `POST /import/json`
  - `POST /import/csv`

4.7 Enrollment Orchestration Logic

`app/services/enrollment.py`:

- validates image payload,
- enforces uniqueness (`candidate_id`, `email`),
- executes recognition pipeline,
- converts quality failures to domain `QualityCheckError`,
- creates candidate and embedding repository records in one request flow.

5. Identification Service (`face_identification`)

5.1 Purpose

Provide real-time 1:N identification by comparing query embeddings against indexed enrolled embeddings.

5.2 Application Startup

`app/core/events.py` startup sequence:

1. load recognition pipeline components,
2. initialize singleton `FAISSManager`,
3. auto-load index from disk if present,
4. if missing/empty, rebuild index from PostgreSQL at startup.

5.3 Recognition Pipeline

`app/recognition/pipeline.py` uses modular components:

- `SCRFDDetector`
- `InsightFaceAligner`
- `ArcFaceEmbedder`
- `CompositeQualityChecker`

It returns a `PipelineResult` with embedding and per-stage timings similar to enrollment.

5.4 FAISS Indexing and Search

`app/faiss/manager.py` responsibilities:

- singleton, thread-safe index manager,
- load/save index metadata from disk,
- in-memory atomic index swap after rebuild,
- rebuild index from DB records (`Candidate` + embeddings),
- expose status and size.

`app/faiss/search.py`:

- normalizes query vectors,
- performs top-K nearest search,
- maps FAISS offsets to candidate UUIDs,
- returns ranked similarity scores.

Configured FAISS options (`app/config/settings.py`):

- index type (`IndexFlatIP`, `IndexIVFFlat`, or `HNSW`),
- index file path,
- default similarity threshold and `top_k`.

5.5 Identification API Surface

- `GET /health`: DB + pipeline + index health diagnostics.
- `POST /identify`: upload image and identify candidate.
- `POST /camera/identify`: camera frame optimized identification path.
- `POST /import/json` and `POST /import/csv`: ingestion endpoints.
- `POST /import/postgres`: sync index from current Postgres records.
- `POST /index/rebuild`: manual FAISS rebuild trigger.
- `GET /index/status`: index status and metadata.
- `GET /metrics`: telemetry and processing statistics.

5.6 Match Decision Logic

Identification flow in `app/api/endpoints/identify.py`:

1. validate file type and size,
2. decode image,

3. run face pipeline,
4. query FAISS for top-K matches,
5. compare top similarity score with threshold,
6. load candidate metadata from DB for confirmed match,
7. return either matched candidate details or `"Unknown Candidate"`.

6. Configuration and Environment

Both services use Pydantic settings loading from `.env`:

- database credentials and host/port,
- model pack path (`buffalo_l`),
- rate limits and log level,
- image constraints (`max_image_size_mb`, allowed content types),
- service port (`8000` enrollment, `8001` identification).

Identification adds FAISS-specific settings:

- index type/path,
- similarity threshold,
- default top-K.

7. Web UI Applications (Streamlit)

7.1 Enrollment UI (`enrollment_service/streamlit_ui`)

Pages include:

- `Home.py`: dashboard and service overview,
- `pages/1_Enroll.py`: candidate enrollment workflow,
- `pages/2_Candidates.py`: candidate list and management,
- `pages/3_Embeddings.py`: embedding operations,
- `pages/4_Export_Import.py`: dataset exchange workflows.

7.2 Identification UI (`face_identification/streamlit_app`)

Pages include:

- `Home.py`: identification service dashboard,
- `pages/1_Identify.py`: live/interactive identification,
- `pages/2_Import_Index.py`: import and index control operations.

8. Operational Guidance

8.1 Local Setup

Per service:

1. create and activate `.venv`,
2. install `requirements-dev.txt`,

3. copy `.env.example` to `.env`,
4. run migrations (`alembic upgrade head` where applicable),
5. start FastAPI app via uvicorn.

8.2 Data Lifecycle

- Enrollment writes source-of-truth candidate and embedding data to PostgreSQL.
- Identification index can be rebuilt at startup or on-demand from PostgreSQL.
- Enrollment export endpoints provide JSON/CSV interchange for external transfer.

8.3 Error Handling and Resilience

- Domain-specific exceptions mapped to structured JSON errors.
- Request-rate limiting mitigates abusive traffic bursts.
- Startup degrades gracefully when models fail to preload (logged, non-crashing behavior).
- FAISS rebuild failures are reported and avoid unsafe partial state.

9. Testing and Quality Tooling

Both services include:

- pytest suites (`tests/`),
- linting with Ruff,
- formatting with Black,
- optional type checks with MyPy,
- coverage targets in `pyproject.toml`.

10. Security and Performance Notes

- CORS is currently permissive (`*`) and should be tightened for production deployments.
- Similarity thresholds should be tuned with representative validation datasets.
- Embeddings are biometric data; encryption-at-rest and access control should be enforced in production.
- CPU inference is configured by default (`CPUExecutionProvider`); GPU acceleration can be evaluated for higher throughput.

11. Current Limitations and Future Improvements

- No explicit authentication/authorization layer on APIs.
- Enrollment and identification are separate services but share a database contract; schema governance should remain strict.
- Operational observability can be expanded with external metrics/log aggregation.
- Index freshness depends on rebuild/sync cadence; event-driven sync could reduce staleness.

12. Conclusion

The codebase is a practical, modular implementation of a full enrollment-to-identification face recognition pipeline. It combines production-oriented API design, deterministic CV/ML processing, persistent vector storage, and scalable vector search. With hardening around security controls and deployment observability, this architecture is suitable for

enterprise-grade 1:N identification workflows.